

Engineering and Validating Unchessed: A Production Research Guide for an Opponent-Adaptive Rust UCI Chess Engine

Unchessed AI Project

Arena.ai Agent Mode

Singapore

Repository: <https://github.com/Amoguslittleahhh/Unchessed-UCI-Engine>

Abstract—This paper documents the engineering, audit, validation, and remaining research program for Unchessed, a Rust Universal Chess Interface (UCI) engine family with a full-strength reviewer and an opponent-adaptive player. The work addresses a difficult combination: chess-rule correctness, search throughput, incremental neural evaluation, strict protocol behavior, reproducible strength controls, conservative opponent classification, human-plausible move choice, complete opening coverage, and production packaging. The implemented foundation includes strict FEN validation; castling, en-passant, repetition, rule-fifty, and transposition-table corrections; shared Lazy SMP node accounting; exact node and root-move limits; incremental HalfKAv2 accumulator state; int16 feature-transformer quantization with AVX-512BW, AVX2, and scalar dispatch; one-Elo posterior granularity from 100 through 3650 with explicit credible intervals; fixed-strength precedence from 100 through 2600; a 3,810-line CCO opening corpus covering all 500 ECO codes; and adversarial UCI, persona, perft, package, and data checks. Local measurements found a 1.537 geometric-mean quantized-NNUE throughput gain and a 1.215 same-tree incremental-accumulator gain. A timing-only human/BOT classifier was rejected: its matched account AUC was 0.413 with a 95 percent account-bootstrap interval of 0.260–0.575. A 48-game real-engine smoke gauntlet produced a descriptive 36.46 percent score against tagged Ethereal, Berserk, and classical Stockfish builds, with intervals too wide for an Elo claim. The paper separates implemented facts from unreplicated historical claims and from external tasks such as policy training, tablebases, high-core scaling, commercial suites, and statistically powered SPRT. It concludes with a staged production roadmap in which every strength-changing intervention is independently measurable and reversible.

Index Terms—chess engine, UCI, Rust, NNUE, SIMD, opponent adaptation, human-like play, engine testing, timing classification, reproducibility

I. INTRODUCTION

A competitive chess engine is often described as move generation, alpha-beta search, and evaluation. A production player-facing engine is substantially broader. It must parse hostile or malformed input without corrupting state, honor asynchronous protocol limits, distinguish game history from search history, manage shared work across threads, load the intended model on a clean installation, remain reproducible under test, and avoid behavior that is socially unsafe or statistically unjustified. An adaptive engine adds a second

control problem: it must estimate current playing strength and select a style without allowing uncertain identity inference to bypass explicit user settings.

Unchessed is organized around the Universal Chess Interface standard [1] as a shared Rust core and three binaries: the adaptive player, the non-adaptive reviewer, and the data generator. The central design decision is that adaptation changes move selection and policy, not the underlying chess rules. The reviewer therefore acts as the fixed reference for search, benchmark, and analysis work. The adapter adds opponent evidence, persona state, human-error matching, book gating, and deterministic randomness controls.

This paper serves two purposes. First, it is an engineering report of the concrete implementation and local measurements. Second, it is a research guide for the remaining program. It does not collapse fixed-depth node reductions into Elo claims, does not call one-Elo bucket spacing one-Elo accuracy, does not infer human identity from missing BOT metadata, and does not report proprietary or unavailable tests as completed.

The principal contributions are:

- a rule-correct and adversarially tested board/UCI foundation;
- a validated incremental and quantized NNUE execution path;
- a conservative strength, identity, and persona architecture;
- full named-opening ECO coverage with risk-graded overlays;
- account-disjoint timing validation across real service exports;
- pinned real-engine game artifacts and provenance; and
- an explicitly gated roadmap for hot-path, policy, reviewer, and release work.

II. RESEARCH QUESTIONS AND EVIDENCE RULES

The project is guided by six research questions.

RQ1: Correctness. Can every externally reachable state obey chess rules and UCI semantics, including malformed

positions, special moves, repetition, rule-fifty, and asynchronous stop conditions?

RQ2: Evaluation throughput. Can neural evaluation approach practical HCE throughput without changing tree scores unexpectedly?

RQ3: Search quality. Which pruning and ordering interventions reduce time-to-depth or improve match strength, rather than merely reducing fixed-depth nodes?

RQ4: Strength control. Can fixed and adaptive modes remain semantically distinct, stable across clock tiers, and honest about the engine ceiling?

RQ5: Opponent modeling. Can move-quality evidence support conservative current-strength decisions while timing remains a non-triggering modulator?

RQ6: Production evidence. Can each claim be reproduced from committed code, checksums, reports, and raw-enough artifacts?

Evidence is classified as follows. **implemented** means code and local regression evidence are present. **gated** means code exists but remains default-off pending game evidence. **external** means required assets or compute are not available in the repository. No feature moves from gated to default-on solely because it lowers nodes at fixed depth.

III. SYSTEM ARCHITECTURE

A. Crate and Binary Boundaries

The unchessed-core crate owns board representation, move generation, FEN/SAN handling, evaluation, NNUE inference, transposition tables, search, openings, policy inference, adaptation, and UCI plumbing. The unchessed-adapter and unchessed-reviewer binaries are intentionally thin wrappers. The reviewer always requests raw strength. The adapter exposes options such as Adaptive, UCI_LimitStrength, UCI_Elo, Troll, BookDepth, PolicyFile, UCI_Opponent, and RandomSeed.

B. Orthogonal Control State

Opponent handling is represented as four conceptual axes:

- strength policy: full, fixed Elo, or adaptive posterior;
- opponent class: human, known engine, unknown computer, suspected engine, or uncertain;
- tactical situation: normal, convert, defend, or clinch; and
- style policy: human, forcing, simplifying, or venomous.

A style cannot override strength policy. In particular, UCI_LimitStrength=true has absolute precedence over persona state.

C. Production Invariants

The implementation enforces several invariants: unknown identity cannot enable automatic trolling; known/computer identity and observed playing strength are separate; a limited Stockfish may be matched near its declared strength while remaining anti-troll locked; full-strength mode has

neutral draw value; all randomness can be seeded; and every positive engine classification is game-latched to prevent threshold flapping.

IV. BASELINE AUDIT AND FAILURE MODES

The initial production audit reproduced protocol and state failures before applying optimizations. Malformed FEN could authorize castling without the required rook and could later panic a worker on an empty from-square. searchmoves was ignored. go nodes 1 could exceed the requested budget by more than three orders of magnitude. go infinite could return when mate was found without waiting for stop. A clean release silently used HCE if the NNUE sidecar was not copied beside the executable. Short move-time probes were dominated by preprocessing and non-incremental evaluation.

A separate persona audit exercised 7,600 direct state combinations. It found that fixed Elo could be bypassed by persona modes, UCI_Opponent metadata was erased by ucinevgame, stale engine priors could survive identity changes, low-clock observations were discarded, full strength still used contempt, and timing logic had the wrong qualitative sign. Historical exhibition logs contained persona mode changes but no PUNISH or CLINCH decisions, so they could not validate the intended policy.

The evaluation audit measured NNUE as slower than HCE on every position in a twelve-position suite. The research baseline reported a 5.27 geometric-mean slowdown, with a 4.30–6.50 range, and traced the dominant cost to full accumulator reconstruction and heap allocation. This established the optimization order: evaluation state first, generic move-generation micro-optimization later.

V. BOARD AND RULE CORRECTNESS

A. Strict FEN Validation

The parser now rejects malformed ranks, invalid piece characters, duplicate or impossible castling rights, invalid en-passant metadata, adjacent or duplicate kings, back-rank pawns, and invalid counters. Four-field FEN remains accepted with strict defaults for counters. Invalid input produces an information string and preserves UCI liveness rather than panicking a search worker.

B. Castling and En Passant

Castling generation independently verifies the king and rook on their home squares in addition to rights and attack constraints. Internal hashing canonicalizes en-passant state when no pseudo-capture exists; FEN serialization retains the supplied legal target. One known limitation remains: the canonical check is pseudo-legal, so a pinned en-passant pawn may distinguish hashes even when no legal capture exists. Fully legal canonicalization is a later board-correctness batch.

C. Repetition and Rule Fifty

The prior repetition test declared a draw on the second occurrence, contrary to the governing Laws of Chess [2]. The correction adds a root-path boundary. A recurrence created inside search may cut immediately, while a position inherited from pre-root game history needs two previous occurrences. Scanning is bounded by the halfmove clock and applies in both main search and quiescence. Checkmate is resolved before rule-fifty and repetition draws. Null moves no longer advance the game halfmove clock. Transposition keys include rule-fifty context so positions with different draw horizons cannot share an unsafe value.

D. Perf Evidence

The release test matrix reproduces start-position depth six at 119,060,324 nodes and Kiwipete depth five at 193,690,690 nodes. Supplementary special-position tests cover promotions, en passant, checks, pins, castling, and legal-move edge cases.

VI. UCI, TIME, AND PARALLEL SEARCH CORRECTNESS

The UCI loop now supports root searchmoves, mate limits, deterministic seeds, shared node limits, go infinite, and basic ponder handling. Infinite search waits for stop. Tiny node limits are checked against one shared atomic counter across Lazy SMP helpers, not against independent local counters. Model and book preprocessing time is charged to the move deadline. Transposition-table generations support aging, and hashfull reports current-generation occupancy.

Adversarial black-box tests verify malformed-FEN rejection and continued liveness, exact tiny node limits, shared two-thread limits, root-move restriction, infinite-stop behavior, and ponder termination. The current ponderhit behavior terminates ponder and emits a move; a true transition into a newly timed continuation remains open work.

VII. NNUE EXECUTION ARCHITECTURE

A. Shipped Network

The shipped design follows the efficiently updatable neural-evaluation lineage [3]. Its file is version 3, HalfKAv2_hm, with 22,528 inputs, a 256-element accumulator, and 5,767,937 parameters. Its exact f32 file size is 23,071,768 bytes and its SHA-256 is recorded by the package and inspection tools. The repository inspection reports no all-zero feature rows.

B. Quantization and Dispatch

At load time, feature-transformer weights and bias are validated for finite values and quantized to signed 16-bit integers using scale 511. Active feature indices are written into a fixed stack buffer rather than a heap vector. Accumulation is selected once at load time among AVX-512BW, AVX2, and scalar backends. The host selected int16-avx512bw. Bounded unsafe blocks are confined to architecture-specific intrinsics. A scalar reference test requires exact accumulator agreement with the selected vector backend.

TABLE I
LOCAL NNUE MEASUREMENTS; NO ELO INFERENCE.

Intervention	NPS effect	Move agreement	Score/tree relation
int16 plus SIMD vs. f32 path	1.537x geometric mean	11/12	quantized scores may drift
Incremental vs. full refresh	1.215x geometric mean	12/12	exact nodes and scores
LMP at depth 10	node ratio 0.651	11/12	pruning diagnostic only

The scalar output head remains f32. This limits reassociation drift while capturing most feature-transformer speedup. The runtime quantized table occupies approximately 11.53 MB.

C. Incremental Accumulators

Evaluation state is ply-indexed through root search, negamax, quiescence, ProbCut, and null moves. Ordinary moves subtract removed HalfKA rows and add inserted rows. A perspective refreshes when its own king changes a bucket or orientation. Castling, promotions, en passant, captures, king movement, null moves, and a depth-three legal tree are compared against full refresh for exact accumulator identity.

D. Measured Effects

Table I separates interventions. The quantized/SIMD comparison used the committed twelve-position harness at depth ten. The incremental comparison temporarily replaced only the update operation with full refresh, preserving the tree.

The research paper that motivated quantization reported mean score drift of 2.5 centipawns and maximum drift of 23 centipawns. The local implementation confirms speed and legal completion but does not claim a match-strength gain. Quantization, incremental evaluation, and LMP each require independently powered matches before Elo attribution.

VIII. SEARCH EXPERIMENTS AND TRANSPOSITION TABLES

Internal iterative reduction, history gravity/malus, counter-moves, razoring, and late-move pruning are implemented behind default-off UCI options. Every option completes legal searches in regression tests. Corrected fixed-depth ratios from the research study were approximately 0.999 for IIR, 1.083 for history gravity, 1.052 for counter-moves, and 1.040 for razoring at depth twelve. LMP reduced nodes more strongly, but fixed-depth node count cannot rank a pruning heuristic because a smaller tree may be weaker or may reach a given depth sooner.

Current TT entries store age/generation, depth, score, move, and bound information. Rule-fifty context is mixed into the key. Remaining TT research includes four-entry clusters, replacement scoring over depth/generation/bound, quiescence probe/store, and one-megabyte stress behavior.

High-priority unimplemented search work includes a staged move picker, main-search SEE pruning, correction

history, continuation history, singular extensions, and deeper reference benchmarking. Each strength-changing feature should receive its own commit, default-off option, fixed-node diagnostics, and match gate.

IX. OPPONENT STRENGTH AND PERSONA CONTROL

A. Identity Versus Playing Strength

OpponentDescriptor persists across new games while per-game observations reset. It stores declared agent type, name, title, rating, known-engine identity, and known-engine strength prior. Known/computer identity is an anti-troll fact; current playing strength is separately inferred.

B. Posterior Granularity and Uncertainty

The detector stores one bucket for every integer Elo from 100 through 3650, for 3,551 probabilities. This is output granularity, not a one-Elo accuracy claim. Decisions use lower and upper credible bounds. Fixed-strength UCI honestly advertises 100 through 2600 because 2600 is the engine ceiling used by the selector.

The live point estimate retains up to 32 informative strength samples and reports their 20th percentile. The recent 75th percentile provides ceiling evidence. Volatility widens uncertainty. The posterior uses broad likelihoods for book and forced moves and stronger evidence for difficult choices.

C. Human-Error Matching

MATCH no longer seeks a narrow and often impossible blunder band. It draws intended error magnitude from a lognormal distribution with sigma 1.1 and human ACPL mean encoded as:

$$\text{ACPL_human}(E) = 300 * \exp(-E / 900).$$

A Gaussian kernel selects among common-depth root alternatives by proximity to intended loss, then blends human-policy priors. Targets at or above the 2600 engine ceiling choose the best move rather than retaining an artificial handicap. A deterministic confidence-scaled oscillation is zero-mean over its cycle.

D. Persona State Machine

Fixed Elo always selects MATCH. Otherwise the state machine prioritizes defense when clearly worse, full strength for unrestricted or detected engines, conversion after a fresh tactical error or stable large lead, and CLINCH only for confidently human, late, queen-rich, drawish positions. CLINCH is suppressed for uncertain and engine opponents. Hysteresis prevents FULL/DEFEND and conversion modes from flapping. One-move blunder freshness is consumed after a decision.

TABLE II
REAL-SERVICE ACCOUNT-LEVEL TIMING STRESS RESULTS.

Service/class	Accounts	Median ACF1	Share at 0.45
Lichess BOT	39	0.212	20.5%
Lichess unmarked	2,141	0.310	30.4%
Chess.com	1,733	0.169	14.4%
unmarked			
FICS unmarked	1,225	0.159	13.7%

X. TIMING CLASSIFICATION: NEGATIVE EVIDENCE

A. Feature and Safety Rule

The runtime feature is lag-one correlation of the log fraction of clock used. For observations $x[0] \dots x[n-1]$, the implementation computes Pearson correlation between $x[0:n-1]$ and $x[1:n]$. It requires at least six positive-duration observations and retains at most 32. A value of 0.45 was inherited as a regularity threshold.

Timing is prohibited from triggering classification by itself. It may only lower the evidence requirement for an opponent already playing at the measurement ceiling. Weak regular play and fast premoves cannot set the engine latch.

B. Account-Disjoint Lichess Validation

The committed snapshot is derived from public Lichess exports [5] and contains 390 valid BOT perspectives from 39 affirmative BOT-title accounts and 2,204 unmarked perspectives from 2,141 accounts. BOT and unmarked records are matched by exact time control and nearest rating within a 200-point cell. Account contributions are capped; the primary statistic aggregates to one median score per account. Bootstrap and permutation units are accounts, not moves.

The matched set has 42 perspective pairs, 20 BOT accounts, and 42 unmarked accounts, with median absolute rating difference two points. Higher timing correlation produced AUC 0.413 with account-bootstrap 95 percent interval 0.260–0.575 and one-sided permutation probability 0.864. At threshold 0.45, sensitivity was 0.200 and the unmarked threshold share was 0.333. Every configured standalone gate failed except source account count.

Chess.com [11] and FICS [12] are threshold stress tests, not verified-human false-positive studies: their exports lack enough affirmative BOT labels. Their account-centered samples nevertheless show that ordinary service traffic frequently crosses 0.45, especially in blitz. The correct conclusion is rejection of timing-only classification, not a reversed classifier.

XI. OPENING COVERAGE AND SAFETY

The original embedded book held 60 lines and covered 54 of 500 ECO codes. The production book now imports 3,810 named lines from the CC0 lichess-org/chess-openings corpus [6] pinned to a specific commit, covering A00 through E99. Curated overlays add 45 serious lines and 15 risk-graded troll lines. Offbeat historical first-move families use a random tier rather than being mislabeled as trolling.

TABLE III
EFFECTIVE MAXIMUM BOOK DEPTH.

Opponent estimate	Maximum plies
100–799	6
800–1199	8
1200–1599	10
1600–1999	12
2000–2399	16
2400+, known engine	40

Effective book depth scales with opponent strength, as shown in Table III. The maximum BookDepth defaults to 40 so strong opponents can reach the full named corpus. CLINCH suppresses troll tiers even under explicit troll mode. A bail-out guard evaluates the live position before continuing a troll line that may already be refuted.

External Polyglot popularity weights are supported, but rating- and time-control-specific empirical frequencies have not yet been generated. Named coverage is not equivalent to popularity calibration.

XII. HUMAN POLICY STATUS

Pure-Rust policy inference and training tools exist, but the repository contains no trained unchessed-maia.bin. Production therefore uses heuristic priors unless an external sidecar is supplied, and logs that fallback. The paper makes no claim that current production moves are generated by a trained Maia policy.

The proper next architecture extends the Maia-style behavioral modeling program [4] into one shared policy conditioned continuously on target Elo, time control, phase, and residual style state, with frozen player-disjoint and future-month holdouts. Training requires licensed data, model provenance, checksums, and account-disjoint calibration. The model should be evaluated on overall move prediction, castling, en passant, promotion, rating-band calibration, and behavior under sparse tails.

XIII. BALANCED DATA AND PRIVACY

The manifest builder samples exact integer-Elo cells from 100 through 3650 crossed with time control, color, and result. It applies per-cell and per-player reservoirs without duplicating sparse records. Position extraction should subsequently balance phase, difficulty, tacticality, and policy entropy. Sparse tails must widen intervals rather than borrow false precision.

Human and engine corpora remain separate. Hard negatives should include titled humans in preparation, weak and limited engines, Maia-style bots, randomized-delay engines, time scrambles, and changing-strength profiles. Evaluation splits must be player-disjoint, game-disjoint, engine-family-disjoint where appropriate, and future-month held out.

Timing records committed to the repository are pseudonymous aggregates; raw PGNs and account names are omitted. Source commits and SHA-256 manifests permit auditing without redistributing large service archives. Commercial or private archives are not scraped merely to fill a table.

TABLE IV
REAL-ENGINE RESULTS FROM THE UNCHESSED PERSPECTIVE.

Opponent	W-L-D	Score	Pair bootstrap 95%
Ethereal 14 HCE	6-9-1	40.62%	21.88–62.50%
Berserk 4.7	5-10-1	34.38%	6.25–65.62%
Stockfish 12 classical	4-9-3	34.38%	15.62–56.25%
Combined	15-28-5	36.46%	descriptive only

XIV. REAL-ENGINE GAUNTLET

The reviewer was tested in 48 actual paired UCI games, not self-play, using Fastchess [8] and pinned Ethereal [9], Berserk [10], and Stockfish [7] sources. Each opponent received the same 20,000-node per-move budget, one thread, 64 MiB hash, MultiPV one, eight opening pairs with colors reversed, and no tablebases. Fastchess concurrency was one on a one-physical-core host. Full PGNs, build commits, executable hashes, opening hash, and adjudication settings are committed.

Median reported Unchessed throughput was about 0.89–0.91 million nodes/s in these games, versus 1.54 million for classical Stockfish 12, 1.76 million for Ethereal, and 1.93 million for Berserk. Cross-engine NPS is not a strength-normalized metric because one node represents different work.

The gauntlet proves protocol interoperability and creates a smoke baseline. Eight independent pairs per opponent cannot establish Elo. Most games use conservative adjudication. Stockfish 18 source compiled, but its official NNUE assets were unavailable from the sandbox network host, so it correctly refused to search. The report uses labeled Stockfish 12 classical mode rather than substituting an unrelated network.

XV. VERIFICATION AND PRODUCTION HARDENING

The combined Rust debug/release matrix contains 96 passing tests, zero failures, and one ignored deep test; the deep perft target passes separately. Python tests cover exact-Elo manifests, timing extraction, variation stripping, pseudonymization, service parsers, snapshot tampering, EPD parsing, and fake-UCI integration. Black-box UCI smoke covers nine normal interactions, while the edge suite covers malformed positions, tiny nodes, shared limits, root restriction, infinite stop, and ponder hit.

Release packaging bundles adapter, reviewer, data generator, NNUE sidecar, and a SHA-256 manifest. It explicitly reports the policy asset as absent with heuristic fallback. CI runs Linux and Windows Rust tests, data checks, model inspection, research benchmarks, UCI/persona smoke, and package generation.

Commercial EPD suites are supported through a coordinate-move runner that records suite and executable hashes, fixed UCI options, per-position nodes/time/depth, and unscored SAN-only answers. This follows established test-position practice [13] while retaining the stronger requirement that game strength be validated separately.

Proprietary positions are not redistributed. Solved percentages are regression diagnostics, not Elo estimates.

XVI. KNOWN LIMITATIONS

The current production state has important limits:

- timing is weak and must never stand alone;
- one-Elo posterior spacing does not imply one-Elo statistical accuracy;
- the fixed-strength ceiling is 2600 although detection spans 3650;
- the policy network asset is absent;
- en-passant hash canonicalization is pseudo-legal rather than fully legal;
- ponder hit does not yet convert into a fresh timed continuation;
- search research switches lack independent SPRT;
- no Syzygy tablebase integration is shipped;
- no high-core 1/2/4/8/16 Lazy SMP sweep has been run;
- the real-engine gauntlet is a smoke-sized HCE-era comparison; and
- explicit project licensing and release signing require owner decisions.

XVII. COMPLETE STATUS MATRIX

Table V is the implementation ledger for the full guide. “Gated” means code exists but is deliberately not a production default. “External” means the repository cannot honestly complete the item without an asset, licensed product, owner decision, or materially different hardware.

Additional performance candidates remain deliberately behind the critical path: precomputed magic constants versus runtime initialization, optional BMI2 PEXT benchmarking, pawn/evaluation caches, smaller reversible board state, and move-generation SIMD only after evaluation/search profiles justify them. Persona candidates include serial error state, bounded threshold jitter, and optional complexity-correlated think time. These are hypotheses, not shipped claims.

XVIII. STAGED RESEARCH ROADMAP

A. Stage A: Search Hot Path

Implement a staged move picker; qsearch TT probe/store; main-search SEE pruning behind a switch; correction and continuation histories; and fixed-node/deeper-reference benchmarks. Record time, nodes, depth, best move, and legal completion. Keep all strength-changing options off until paired matches.

B. Stage B: Board and Move Generation

Profile direct legal generation using check and pin masks. Compare it with current pseudo-legal generation plus filtering. Consider compact reversible make/unmake state only if profile data justifies the complexity. Preserve exact incremental NNUE special-move tests and deep perf.

C. Stage C: TT Architecture

Move to four-entry clusters, define replacement value over depth, age, and bound, add qsearch usage, and stress one-megabyte hash. Validate no rule-fifty leakage and stable current-generation hashfull.

D. Stage D: Policy and Persona

Train a continuous-Elo shared policy with frozen provenance. Add serially correlated error state, bounded threshold jitter, optional complexity-correlated timing, and shadow-only classifier logs. No automatic trolling should depend on timing or uncertain identity.

E. Stage E: External Validation

Run independent SPRTs for quantization, incremental NNUE, and LMP. Test Lazy SMP on 1, 2, 4, 8, and 16 physical cores. Extend the gauntlet to current Stockfish, current Berserk, RubiChess, and an MCTS engine. Use many opening pairs and publish confidence intervals.

F. Stage F: Reviewer and Distribution

Add PGN review, WDL-based move classification, accuracy scoring, and HCE explanations. Integrate optional Syzygy paths without bundling giant tables. Add model IDs/checksums to UCI, release signing, explicit licensing, and platform-specific packages.

XIX. OPERATIONAL REPRODUCTION GUIDE

A clean release build is:

cargo build --release

The NNUE sidecar must be beside the executable or selected with EvalFile. Core verification is:

cargo test --release --workspace --all-targets --all-features

cargo test --release -p unchessed-core
perf::tests::deep -- --ignored

The model inspector validates dimensions, finite weights, expected file size, quantized runtime bytes, and zero rows. Opening coverage is checked independently. UCI smoke should be run against both HCE fallback and NNUE package layouts.

Timing reports are regenerated in check mode from the committed JSONL and manifest. The real-engine report is regenerated from the complete PGNS in benchmarks/real-engines. Licensed coordinate EPD suites can be run through tools/uci_epd_suite.py; the adaptive binary is rejected by default.

For every new search feature, the recommended sequence is:

- 1) add an isolated default-off option and deterministic test;
- 2) prove legal completion, score sanity, and no protocol regression;
- 3) profile representative positions under fixed nodes and fixed time;

TABLE V
CONSOLIDATED IMPLEMENTATION AND EVIDENCE LEDGER.

Area	Item	State	Required next evidence or action
Board/UCI	strict FEN, castling pieces, repetition boundary, rule fifty, searchmoves, shared nodes, infinite stop	Implemented	Maintain adversarial regression and deep perf on every architecture.
NNUE	int16 transformer, SIMD dispatch, stack features, incremental accumulator	Implemented	Independent paired matches before assigning Elo to quantization or incrementality.
Search experiments	IIR, gravity, countermoves, razoring, LMP switches	Gated	Fixed-time diagnostics followed by isolated SPRT; do not promote from fixed-depth nodes.
Search hot path	staged move picker, qsearch TT, SEE pruning, correction and continuation histories, singular extension	Open	Profile, implement one feature per commit, add deterministic legality and match gates.
Board architecture	direct legal generation, compact reversible state, fully legal en-passant hash	Open	Same-tree evaluation checks and full perf matrix.
Transposition table	clustered entries and replacement scoring	Open	Hash-size sweep, quiescence tests, and rule-fifty isolation.
Persona	descriptor persistence, fixed-Elo precedence, anti-troll lock, latches, heavy-tail MATCH	Implemented	Shadow logs and broader fixed-strength game calibration.
Timing	ACF feature and account-disjoint service benchmark	Rejected standalone	Keep modulator-only rule; obtain affirmative humans and millisecond clocks before reconsideration.
Opening	3,810 CC0 lines, 500 ECO codes, overlays, depth scaling	Implemented	Add rating/time-specific popularity frequencies and holdout fingerprint tests.
Human policy	inference/training code	External asset	Train, freeze, checksum, package, and evaluate a continuous-Elo model.
Reviewer	full-strength UCI reference	Implemented core	Add PGN review, WDL classifications, accuracy, and explanation layer.
Endgames	Syzygy probing and distribution	External/design	Implement optional probing; owner selects supported sets and packaging policy.
Parallelism	shared Lazy SMP accounting	Implemented basic	Run 1/2/4/8/16 physical-core scaling and stability sweep.
Production	CI, package manifest, Linux/Windows tests	Implemented basic	Project license, signing keys, release signatures, and model provenance policy.
Commercial validation	service aggregate and licensed EPD runner	Partially implemented	User supplies lawful proprietary exports or runs the reviewer in the licensed GUI.

- 4) run paired color-reversed games with frozen openings and binaries;
- 5) publish hashes, W/L/D, pentanomial counts, and confidence intervals; and
- 6) promote only if the predeclared gate passes.

XX. DISCUSSION

The most consequential lesson is optimization order. When evaluation dominates node cost, generic search-loop micro-optimization has a strict ceiling. Removing allocations, quantizing the feature transformer, and carrying accumulator state produced measurable local speedups while preserving an exact reference path.

The second lesson is that protocol correctness is strength infrastructure. A node budget exceeded by helper threads, an uncharged preprocessing phase, or a draw accepted one occurrence early changes practical playing strength and invalidates benchmarks. Correctness cannot be deferred until after Elo tuning.

The third lesson is that adaptive behavior requires stricter epistemic discipline than raw search. A plausible timing feature can fail across unseen accounts. Missing BOT metadata is not proof of humanity. A 3,551-bucket posterior can still be uncertain. Conservative defaults, anti-troll locks, latches, explicit ceilings, and credible intervals are product requirements, not cosmetic policy.

Finally, reproducibility should be designed into the engine. Deterministic seeds, sidecar checksums, generated reports, pinned external commits, raw PGNs, default-off options, and fixed reviewer settings make negative findings useful. The

timing failure is therefore a successful research outcome: it prevents an unsafe standalone classifier from shipping.

XXI. CONCLUSION

Unchessed now has a substantially stronger production foundation: strict chess state, rule-correct draw handling, adversarial UCI behavior, shared node accounting, incremental quantized NNUE, persistent opponent identity, honest fixed-strength semantics, complete named-opening coverage, reproducible timing validation, and real-engine game artifacts. Local throughput gains are measurable, but new Elo claims remain gated. Timing-only classification is rejected. A trained human policy, tablebases, current-engine gauntlets, high-core scaling, deeper reviewer features, and independent SPRTs remain external or future work.

The roadmap deliberately favors isolated, attributable interventions over a mega-patch. That discipline is the central research guide: fix correctness first, optimize the measured bottleneck, separate identity from strength, treat uncertainty as state, and require game evidence before promoting anything that changes strength.

ARTIFACT AVAILABILITY

The source, manifests, reports, tests, and PGNs described here are organized under unchessed-core, tools, docs, data, and benchmarks/real-engines in the project repository. Large raw service datasets, proprietary suites, model training corpora, commercial binaries, and credentials are intentionally excluded.

REFERENCES

- [1] S. Meyer-Kahlen, “The Universal Chess Interface,” UCI protocol specification, 2006. [Online]. Available: <https://www.shredderchess.com/chess-info/features/uci-universal-chess-interface.html>
- [2] FIDE, “FIDE Laws of Chess,” International Chess Federation. [Online]. Available: <https://handbook.fide.com/chapter/E012023>
- [3] Y. Nasu, “Efficiently Updatable Neural-Network-based Evaluation Functions for Computer Shogi,” 2018. [Online]. Available: <https://github.com/ynasu87/nnue/blob/master/docs/nnue.pdf>
- [4] R. McIlroy-Young, S. Sen, J. Kleinberg, and A. Anderson, “Aligning Superhuman AI with Human Behavior: Chess as a Model System,” in *Proc. KDD*, 2020.
- [5] Lichess, “Lichess Open Database,” CC0 game, puzzle, and evaluation exports. [Online]. Available: <https://database.lichess.org/>
- [6] Lichess, “Chess Openings,” CC0 named ECO corpus. [Online]. Available: <https://github.com/lichess-org/chess-openings>
- [7] The Stockfish Developers, “Stockfish,” GPL-3.0 UCI chess engine. [Online]. Available: <https://github.com/official-stockfish/Stockfish>
- [8] D. Reinfarth et al., “fastchess,” UCI engine match runner. [Online]. Available: <https://github.com/Disservin/fastchess>
- [9] A. Grant, “Ethereal,” GPL-3.0 UCI chess engine. [Online]. Available: <https://github.com/AndyGrant/Ethereal>
- [10] J. Honnold, “Berserk,” GPL-3.0 UCI chess engine. [Online]. Available: <https://github.com/jhonnold/berserk>
- [11] Chess.com, “Published Data API,” public player and game archive interface. [Online]. Available: <https://www.chess.com/news/view/published-data-api>
- [12] FICS Games Database, “Download and move-time documentation.” [Online]. Available: <https://www.ficsgames.org/help.html>
- [13] Chessprogramming Wiki, “Engine Testing and Test Positions.” [Online]. Available: <https://www.chessprogramming.org/Test-Positions>
- [14] Unchessed AI Project, “Unchessed UCI Engine repository and reproducibility artifacts,” 2026. [Online]. Available: <https://github.com/Amoguslittleahhh/Unchessed-UCI-Engine>